

Description du projet dans ses grandes lignes (organisation générale du code)

L'objectif de ce projet était la création d'un jeu de **bataille navale**.

Pour cela, nous disposons de **trois fichiers clés** : `vue.c`, `control.c` et `jeu.h`.

Nous avons utilisé le **modèle du jeu** déjà existant, et notre tâche principale consistait à implémenter la **vue** ainsi qu'un **contrôleur**. Nous avons donc créé un fichier `control.c`, dans lequel nous faisons le lien entre la vue (`vue.c`) et le modèle (`jeu.h`).

Concrètement, le contrôleur permet de **mettre à jour la vue en fonction des interactions du joueur**, tout en respectant les règles définies par le modèle.

Dans les grandes lignes : lorsqu'on exécute le programme, on commence par la **connexion de deux joueurs**, qui passent ensuite à la phase de **placement de leurs navires**. Pour cela, nous avons utilisé les fonctions disponibles dans `jeu.h`, permettant notamment de récupérer l'**adresse IP**, le **pseudo**, etc.

Une fois les bateaux placés, **la bataille peut commencer** :

Le joueur attaque, et sa **grille se met automatiquement à jour** pour indiquer le résultat :

- **"Touché"** : un navire a été atteint
- **"Flop"** : aucune cible touchée
- **"Coulé"** : un navire a été totalement détruit

Ensuite, c'est au tour de l'adversaire. Pendant ce temps, **les boutons du joueur actif sont désactivés**, et ceux de l'autre joueur sont activés. Notre grille quant à elle est mise à jour après avoir récupéré le coup de l'adversaire.

Cela a nécessité **un travail important sur la coordination entre la vue et le modèle**, afin d'assurer une synchronisation fluide entre les actions du joueur et l'évolution de l'état du jeu.

Indication des éventuelles difficultés rencontrées, ainsi que ce que le projet vous a éventuellement apporté :

Démarrer ce projet a été une difficulté en soi. En effet, au début, nous ne savions pas vraiment comment nous organiser. Cependant, nous avons rapidement rétabli nos idées en commençant par les bases. Nous avons donc entamé l'implémentation de la vue. Cela semblait relativement simple au départ, mais les erreurs se produisent rapidement. Comme on dit souvent, "le diable se cache dans les détails" en informatique. Ainsi, un petit moment d'inattention pouvait entraîner des heures de recherche de solutions. Ce projet nous a donc appris à être plus efficaces dans le débogage. Par exemple, nous avons pris l'habitude d'ajouter des printf de manière pertinente pour trouver rapidement les problèmes. De plus, faire plus d'efforts sur la dénomination des fonctions permet une meilleure compréhension du projet et facilite sa lisibilité.

Nous avons notamment été confrontés à un problème qui nous a coûté beaucoup de temps : la vue ne se rafraîchissait qu'une fois toutes les fonctions de rappel associées à un même événement (ici, le clic sur un bouton) terminées. La solution nous est tombée du ciel : "Pour contourner ce problème, plutôt que d'associer tous vos traitements au clic, vous pouvez en associer certains à l'évènement "press-event", et d'autres à l'évènement "release-event" ".

De plus, une erreur au niveau de l'une des fonctions du modèle nous a pris un certain temps à résoudre. En effet la fonction `jeu_est_coup_valide()` ne fonctionnait pas initialement.

Par ailleurs, la compréhension du fichier **jeu.h** nous a posé quelques difficultés, notamment en raison de la découverte du modèle.

Pour conclure, ce projet nous a permis d'améliorer énormément nos compétences et particulièrement au niveau du débogage et de la gestion des fonctions.

Les évolutions que nous envisageons :

À la fin de la partie, nous aurions aimé ajouter une option permettant de se connecter avec un autre joueur.

Autres :

Il n'y avait pas de fonction dans le modèle qui retourne le joueur et le nombre de parties jouées ainsi on a été obligé d'utiliser le contrôleur pour y accéder :

"control->modèle->j" et "control->modèle->nb_de_partie_jouées".